

SC102 L11: UI 系统进阶——详细学习笔记

涵盖: Scroll View | Auto Layout 系统 | UI 自适应 | 游戏暂停 | UI 层级管理 | UI 性能优化

1 Scroll View 滚动窗口

[p.4-6]

Scroll View 是 Unity 用于展示可滚动内容的 UI 系统。它不是单一组件，而是多个组件的组合：
核心组成：

- **Scroll Rect 组件：**可滚动区域的核心——管理 Content 滚动和 Scroll Bar 联动
- **Viewport 物体：**可见区域（配有 Mask 组件防止内容超出）
- **Scroll Bar 组件：**滚动条，最重要的属性是 `value`（0-1 之间的当前位置）

Scroll Rect 关键属性：

- **Content：**滚动的内容（RectTransform）
- **Horizontal/Vertical：**允许哪个方向的滚动
- **Movement Type：**是否有边界限制、是否回弹
- **Scroll Sensitivity：**灵敏度，越高滚动越快
- **Viewport：**可见区域引用
- **Horizontal/Vertical Scrollbar：**关联的滚动条组件

Scroll Bar 关键属性：

- **Interactable：**是否可互动（Button、Slider 等也都有此项）
- **Value：**当前滚动位置（0-1）
- **Size：**可点击滑块的大小
- **Number of Steps：**离散位置数量（0= 连续滚动）

2 Auto Layout 自动排版系统

[p.7-11]

Auto Layout 是 Unity 的 UI 排版解决方案，能应对动态 UI 修改并保持排版一致性。

核心概念：

- **Layout Element:** 拥有 RectTransform 的 UI 物体，有 6 个布局属性：Minimum Width/Height、Preferred Width/Height、Flexible Width/Height
- **Layout Controller:** 利用 Layout Element 数据进行自动排版的控制器

分配优先级：Minimum → Preferred → Flexible。先分配最小空间，有剩余再分配偏好空间，再有多余分配弹性空间。

三种 Layout Controller：

1. **Layout Element 组件：** 手动设置 min/preferred/flexible 值
2. **Content Size Fitter:** 根据子物体内容自动调整自身 Layout Element 大小。选项：Horizontal Fit/Vertical Fit (MinSize/PreferredSize/Unconstrained)
3. **Layout Group:** 控制子物体排列。三种类型：
 - **Vertical Layout Group:** 垂直排列
 - **Horizontal Layout Group:** 水平排列
 - **Grid Layout Group:** 网格排列

Layout Group 关键属性（以 Vertical 为例）：

- **Padding:** 上下左右边距
- **Spacing:** 相邻元素间距
- **Control Child Size:** 是否控制子物体尺寸
- **Child Force Expand:** 有多余空间时是否放大子物体填充

常见组合：Content Size Fitter + Vertical Layout Group → 自动根据内容调整高度的垂直列表。Scroll Rect 读取 Content Size Fitter 调整后的大小来决定何时停止滚动。

3 UI 自适应 (Responsive UI)

[p.14]

不同屏幕 Aspect Ratio 下需要使用 Anchor 来定义各面板组件的位置。Anchor 决定 UI 物体固定在父物体的哪个位置（中间/左上角/右下角/拉伸等）。

关键技巧：使用 Anchor Presets（如 stretch+stretch）使 UI 元素随屏幕尺寸自动缩放。

4 游戏暂停功能

[p.16]

基本方案: `Time.timeScale = 0`

但这会产生问题:

1. **UI 动画失效**: UI Animator 也受 `timeScale` 影响 → 将 UI Animator 的 Update Mode 设为 **Unscaled Time**
2. **玩家输入屏蔽**: 暂停时仍需禁止角色操作 → 用暂停状态标志作为输入判断条件
3. **多层 UI 界面**: 打开一个界面时暂停, 但在暂停中又打开另一个界面 → 需要层级管理

5 UI 界面层级管理

[p.17]

使用 **Stack (栈)** 管理多层 UI 界面 (为什么是 Stack? LIFO——最后打开的界面最先关闭)。

```
public class UIManager : MonoBehaviour {  
    private Stack<GameObject> panelStack = new Stack<GameObject>();  
  
    public void OpenPanel(GameObject panel) {  
        if (panelStack.Count == 0)  
            Time.timeScale = 0;           // 第一个面板: 暂停游戏  
        panelStack.Push(panel);  
        panel.SetActive(true);  
    }  
  
    public void ClosePanel() {  
        if (panelStack.Count == 0) return;  
        var panel = panelStack.Pop();  
        panel.SetActive(false);  
        if (panelStack.Count == 0)  
            Time.timeScale = 1;           // 最后一个面板关闭: 恢复游戏  
    }  
}
```

6 UI 性能优化

[p.19-22]

6.1 1. 动静分离

Canvas 的工作原理：根据子物体中的 UI Element 生成 Mesh 提交 GPU 渲染。Canvas 内任何一个物体变动，整个 Mesh 就需要重新生成（极耗性能）。

解决：静态物体（完全不变化）和动态物体（有动画）放在不同 Canvas 中。每个 Canvas 独立构建 Mesh，变动影响范围变小。

6.2 2. 减少不必要的射线检测

许多 UI 组件默认开启 Raycast Target。每多一个射线检测目标，引擎多做一次检测。不可互动的组件（纯展示文本、背景图等）不需要此选项。

解决：仅在可互动组件勾选 Raycast Target。如果整个 Canvas 都不可互动，直接移除 Graphic Raycaster 组件。

6.3 3. 尽量少用 Layout Group

Layout Group 性能消耗大——其中元素改变时会不断向上寻找有效 Layout Group 来重新调整位置，通过 GetComponent 实现（昂贵）。

解决：在代码中手动计算 Layout 位置（手动排版），避免运行时依赖 Layout Group 的动态计算。

6.4 4. 昂贵的 UI 动画

Animator 迫使其中的 UI 元素每一帧都重新更新，即使动画是静止的。

解决：使用 Tweening 系统（如 L09 的 StartProcedure）在代码中制作 UI 动画。Tweening 只在动画进行时更新，静止时不消耗。

7 课后习题详解

[p.23]

7.1 题 1：水平固定、垂直随文本自适应的对话框

设置：

- 对话框父物体：固定宽度（RectTransform width= 定值），添加 Vertical Layout Group
- 文本子物体：添加 Content Size Fitter（Vertical Fit=Preferred Size）
- 背景 Image：添加 Content Size Fitter（Vertical Fit=Preferred Size），或代码设置 background.rectTransform.sizeDelta

7.2 题 2: Time.timeScale 之外的暂停方法

替代方案——手动暂停管理器:

```
public class PauseManager {  
    public static bool IsPaused { get; private set; }  
  
    public static void Pause() { IsPaused = true; }  
    public static void Resume() { IsPaused = false; }  
}  
  
// 在每个MonoBehaviour的Update中检查  
void Update() {  
    if (PauseManager.IsPaused) return;  
    // 正常逻辑...  
}
```

优点: 不修改全局 timeScale (不影响 Unscaled Time 粒子/动画); 可选择性暂停某些系统 (如只暂停敌人 AI 不暂停 UI); 不受 Animator Unscaled Time 限制。**缺点:** 需要在每个需要暂停的脚本中添加检查, 侵入性较强。

7.3 题 3: UI 界面优化实践

检查清单: 动态 UI 是否和静态 UI 分 Canvas? 不可交互 Text/Image 是否关了 Raycast Target? 是否大量使用 Layout Group (考虑手动排版)? UI 动画是否用 Animator (考虑改 Tweening)?